

Routing3D 자동 배관 라우팅 — 소프트웨어 아키텍처 · 핵심 알고리즘 · 개발 상세

플랜트 배관 3D 직교 자동 라우팅 엔진. 단위 mm · 기본 셀 50mm.

본 문서는 현재까지 개발된 자동라우팅 기능의 전체 구조와 핵심 알고리즘(점유/방문맵, 직교 AW*, 시작/종단 스텝, 그룹배관 패턴 활용, 기존배관 복제)을 설계·구현 관점에서 상세히 정리한다.

1. 개요와 목표

Routing3D 는 PostgreSQL(AUTOROUTINGV7)의 실제 BIM 장애물·메인 장비 PoC(Point of Connection)를

입력으로 받아, 충돌 없는 3D 직교(맨해튼) 배관 경로를 자동 산출하는 엔진이다. 핵심 요구는 두 가지다.

- **물리적 타당성:** 장애물(구조물·설비·덕트)을 관통하지 않고, 배관끼리도 충돌하지 않는다.
- **설계 유사성:** 사람이 설계한 기존 배관(TB_ROUTE_PATH)의 형상·습관(출발/종단 스텝, 공용 트렁크 랙,

유틸리티별 다발)을 학습해, 자동 결과가 "사람 설계처럼" 보이게 한다.

성능 목표는 8,000m 도메인에서 단일 배관 1 초 미만, 전체(수백 본) 1 분 내외, 메모리 32GB 이하다.

1.1 개발 단계 요약

단계	내용
Phase 1	Python 알고리즘 레퍼런스(점유/AW*/비용/다중배관/scene I/O)
Phase 2	인터페이스·계약 동결(불변식 명세)
Phase 3	C++ 엔진 + C ABI DLL + C# WPF 뷰어

학습 확장	pgvector 기반 스텝/그룹배관 패턴 학습·활용(L1~L4)
설계 복제	매칭 기존배관 폴리라인 복제 + 국소 수리

2. 소프트웨어 아키텍처

2.1 3-티어 구조

시스템은 동일한 알고리즘을 세 계층이 1:1 로 미러링하는 구조다. 각 계층은 같은 골든(정답) 셋으로

교차검증되어, 어느 계층에서 보든 동일한 경로·확장수가 나온다.



2.2 계층 분리의 이유

- **Python:** 알고리즘을 빠르게 실험하고 "정답"을 고정한다. 모든 회귀의 기준선.

- **C++**: 성능. 점유맵을 컴파일타임 다형성(템플릿)으로 추상화해, 백엔드(Dense/Sparse/Implicit)를

바꿔도 AW* 코드는 동일하다. 외부 의존성 0 의 단일 DLL 로 빌드된다.

- **C#**: 실제 사용자 환경. DB 로드, 3D 시각화, 학습 데이터 활용, 헤드리스 진단을 담당한다.

2.3 인터럽(C ABI) 안전 규칙

C++ 엔진은 C ABI(`extern "C"`)로 노출되어 C#이 P/Invoke 로 호출한다. 경계 규약은 엄격하다.

- 예외는 ABI 경계를 넘지 않는다 — 모든 네이티브 함수는 `try/catch` 로 감싸 상태코드(R3dStatus)를 반환.
- 모든 문자열은 UTF-8, 구조체는 POD(blittable), 호출규약 cdecl, 비트 x64 고정.
- 핸들은 불투명 포인터(R3dEngine*)로 관리하고, C#은 `SafeHandle`(R3dEngineHandle)로 수명을 보장.

2.4 핵심 불변식

불변식	내용
AW* 결정성	(f, 삽입순서 counter) tie-break + 고정 이웃순서 → 동일 입력 → 동일 경로/확장수
다중배관	성공 경로 셀 공유 0(충돌 0), 원본 점유맵 불변(<code>copy()</code> 사본 사용)
백엔드 무관	Dense/Sparse/Implicit 어느 점유 백엔드든 동일 결과
scene.txt	바이트 무손실 왕복(<code>format_repr_double = Python repr(float)</code>)

3. 입력 데이터와 격자 구성

3.1 PostgreSQL 입력

테이블	용도
space_project_map	프로젝트 목록
TB_BIM_OBSTACLES	장애물 AABB(min/max XYZ, mm)
TB_BIM_EQUIPMENT	메인 장비 + PoC 목록(POC_LIST jsonb)
TB_DUCT_LATERAL	종단 객체(덕트/레터럴)
TB_ROUTE_PATH	기존 설계배관 폴리라인(패턴 학습 입력)
route_stub_pattern / route_bundle_template	학습 저장소(pgvector)

3.2 작업(Task) 생성

각 PoC 는 {id, pocPosition, utility, utilityGroup, endPocs:[...]} 구조이며, 연결된 PoC 쌍(시작 장비 PoC

↔ 종단 덕트/레터럴 PoC)이 하나의 **라우팅 작업**(start_mm → end_mm)이 된다. 기본은 isConnected=true 만 작업화.

3.3 격자(Grid)

- 원점 origin = lo(장애물 BBOX 최소), 형상 shape = ceil((hi-lo)/cell).
- 셀 크기는 정밀도/속도 트레이드오프: 50mm(빠름·표준), 25mm·10mm(정밀·거대격자).
- 셀 크기가 작을수록 격자 셀 수는 세제곱으로 늘어난다(예: cell=50 → 약 11M 셀, cell=25 → 약 90M 셀).

4. 핵심 자료구조 — 점유맵과 방문맵

4.1 점유맵(Occupancy) — 3 종 백엔드

점유맵은 "이 셀이 장애물로 막혔는가"를 답하는 자료구조다. 셀 크기에 따라 메모리·속도가 달라

3 가지 백엔드를 두고, C++ 템플릿으로 AW* 코드와 분리했다(컴파일타임 다형성).

백엔드	메모리	용도
DenseOccupancy	O(전체 셀) 1 바이트/셀	소~중 격자(골든.정답 기준). cell=50 11MB
SparseOccupancy	O(막힌 셀) 해시	계층 corridor(초대형 격자)
ImplicitOccupancy	O(장애물 수)	정밀·거대격자. 복셀화를 폐기하고 장애물 AABB 를 SpatialBoxIndex(유니폼 그리드)로 색인해, 셀 질의 시 박스 최근접 검사로 즉석 판정. 격자 크기와 무관한 메모리

핵심 설계: cell≤25mm 의 거대격자(수억~수십억 셀)에서 Dense 점유맵은 메모리 한계에 부딪힌다.

ImplicitOccupancy 는 복셀맵 자체를 만들지 않고 "장애물 박스 색인 + 온디맨드 질의"로 전환해,

셀 크기를 줄여도 메모리가 늘지 않는다. capi 가 격자 크기 임계 초과 시 자동 전환한다.

4.2 클리어런스(Clearance)

배관이 장애물에 너무 붙지 않도록, 막힌 셀로부터의 BFS 거리변환을 비용에 반영한다.
거대격자에서는

전역 거리맵(배관당 size×4B BFS) 대신 **온디맨드 클리어런스**(박스 최근접 질의)로 계산해 메모리를 아낀다.

4.3 방문맵(Visited Map)

AW* 가 탐색 중 **확장(expand)**한 셀의 집합이다. 경로 탐색의 부산물로, 두 용도가 있다.

- **진단·시각화:** 실패한 배관이 "어디까지 탐색하다 막혔는지"를 노란 큐브로 보여준다(혼잡/막힘 구분).
- **단계별 애니메이션:** 확장 순서대로 표시해 시작 PoC → 종단 PoC 로 나아가는 과정을 재생한다.

방문맵 수집은 토글 가능(`SetCollectVisited`)하며, OFF 면 라우팅 후 방문 셀이 비어 메모리/시간을 아낀다.

5. 핵심 라우팅 알고리즘 — 직교 AW^*

5.1 상태 정의와 직교 이동

배관은 직교(축정렬)로만 꺾이므로, AW^* 상태는 단순 셀이 아니라 **(셀, 진입 방향)** 쌍이다.

- 이동: $\pm x, \pm y, \pm z$ 6 방향(6-연결).
- **꺾임 비용(w_{turn})**: 진입 방향과 다른 방향으로 나가면 가산 $\rightarrow$ 불필요한 지그재그 억제.
- 상태에 방향을 포함해야 "직진 중 vs 방금 꺾음"을 구분해 꺾임 비용을 정확히 매긴다.

5.2 비용 모델

경로 비용은 **가산 페널티의 합**으로 설계해 휴리스틱 허용성(admissibility)을 보존한다.

```
cost(step) = 이동거리(셀)
              + w_turn    × (방향 전환 여부)
              + w_clear   × clearance_penalty(셀)      # 장애물 근접 회피
              + w_corridor × (회랑 밖 셀 여부)          # 설계 추종(소프트 바이어스)
              - (rack_levels 면제 z-셀이면 w_corridor 가산 면제)
```

모든 항이 **가산(빼지 않음)**이라 휴리스틱이 비용을 과대평가하지 않는다(최단성 보존).

5.3 결정성(determinism)

동일 입력에 항상 동일 경로가 나오도록:

- 우선순위 큐 tie-break = **(f 값, 삽입순서 counter)** — f 가 같으면 먼저 들어온 노드 우선.
- 이웃 확장 순서를 고정.

$\rightarrow$ 부동소수 비결정성 없이 재현 가능. C++ ctest 가 골든 셋으로 확장수까지 일치 검증한다.

5.4 가중 AW*(weighted) + 동적 수렴 가중

표준 AW*(휴리스틱 가중 1.0)는 최단이지만, 솔리드 설비·덕트가 많은 혼잡 격자에서 탐색 폭이 커진다.

실데이터는 가중 AW*(w_heur=2.0, 목표지향 그리디)로 탐색 상한(12M 확장) 내에 어려운 우회를 찾는다.

그러나 정적 w_heur=2.0(공격적 그리디)은 목표/PoC 근처 혼잡·막다른길에서 함정에 빠져 마지막 접근

경로를 놓친다. 이를 동적(수렴) 가중으로 해결했다.

```
w_eff = w_heur_near + (w_heur - w_heur_near) × (h / h_start)
    먼 곳(h≈h_start) = w_heur(2.0, 빠름)
    목표 근처(h→0)   = w_heur_near(1.0, 표준 A*, 정확)
```

준최적 상한은 여전히 w_heur. 핵심 게이트: 무제한 탐색(max_expansions≤0)에서만 적용
— 예산-게이트

(probe 300k·hier)에서 목표근처 가중을 낮추면 escalation 폭주하므로 거대격자에선 자동 비활성.

*실측(project6 cell=100): 스텝 ON 시 동적 가중으로 206→208(완전 성공), 스텝 OFF
199→203,*

시간 동일. w_heur=1(골든)이면 동적 비활성이라 골든 정확도 불변.

5.5 임계 자동 결정

```
weighted = (전체 셀 수 > 300,000)
w_heur    = weighted ? 2.0 : 1.0   (작은 데모 격자는 표준 A*)
w_heur_near = weighted ? 1.0 : 0.0
```

6. 다중 배관 라우팅 — 충돌 0 보장

6.1 순차 + mark_pipe

여러 배관을 한 격자에 깔 때 서로 겹치지 않아야 한다. 전략은 우선순위 순 순차 라우팅 + mark_pipe 다.

1. 우선순위(긴 배관 우선 **longest** 또는 유틸리티별 **utility**)로 작업 정렬.
2. 한 배관 라우팅 성공 → 그 경로 셀을 점유맵에 **mark_pipe**(추가 점유)로 등록.
3. 다음 배관은 이미 깔린 배관을 장애물로 보고 회피 → **충돌 0**.

원본 점유맵은 **copy()** 사본에만 마크해 불변을 유지한다(불변식 M1·M2).

6.2 rip-up & reroute

순차 라우팅은 먼저 깔린 배관이 길을 막아 나중 배관이 실패할 수 있다. **rip-up** 은 실패 시 충돌하는

기존 배관을 걷어내고(rip-up) 순서를 바꿔 다시 깐다(reroute). 무손실·결정적으로 동작한다.

합성 혼잡(9×9 벽+틈)에서 순차 1/2 → rip-up 2/2, project6 cell=200 에서 multi 77 → rip-up 80(+3).

6.3 계층(hierarchical) corridor

cell=10mm 같은 거대격자에서 직접 AW*는 탐색량이 폭증한다. **계층 corridor** 는 coarse(거친) 격자에서

가이드 경로를 먼저 찾고, fine(세밀) 격자에서 그 통로(±N 셀) 안만 탐색한다(escalation 게이트).

- 저예산(300k 확장) 직접 AW*를 먼저 시도 → 초과하는 '어려운 배관'만 hier 로 승격.
 - 쉬운 배관(개방 랙 직선)은 직접 AW* 그대로라 cell≤25 회귀 0·품질 바이트 동일.
-

7. 기존설계 학습과 활용

자동 결과를 "사람 설계처럼" 만들기 위해, 기존 배관(TB_ROUTE_PATH)에서 형상 습관을 학습해 라우팅에

주입한다. 학습은 Python(pgvector)에서 하고, 활용은 C# 뷰어(엔진 파라미터/시작·끝점 보정)에서 한다.

7.1 시작/종단 스텝(Stub) — 수직배관 + 첫 엘보

사람이 설계한 배관은 장비에서 수직으로 빠져 첫 엘보로 수평 전환한 뒤 공용 랙을 타고, 종단 객체

앞에서 다시 수직으로 진입한다. 이 양 끝의 짧은 정형 구간이 스텝이다.

스텝 추출 알고리즘(`StubExtractor`, Python `_walk_stub` 포팅):

4. **방향 런 압축**: 폴리라인을 같은 방향이 이어지는 '런'으로 묶는다.
5. **짧은 런 흡수**: 250mm 미만의 짧은 런(측정 지터)은 인접 런에 흡수한다.
6. **엘보 탐지**: 첫 수직($-z/+z$) 런 + 그 뒤 첫 수평 전환(엘보)까지를 스텝으로 절단(`STUB_LEADIN_MM=800`).
7. **점열 절단**: 스텝 끝 = 랙 위 자유공간의 시작점.

예) 장비(EQUIP) 스텝 $-z, +x$ (아래로 빠져 $+x$ 로 꺾음), 덕트(DUCT) 스텝 $+z, -x$. 엘보 방향이 학습 특징에

인코딩된다(`vector(24)` 특징 + `vector(3)` 방향단위, HNSW 색인).

스텝 라우팅(`UseStubRouting`, 기본 ON):

- 매칭되는 기존배관의 출발/종단 스텝을 고정 설계 구간으로 깔고, AW^* 는 스텝 끝 ~ 스텝 끝(랙 위

자유공간)만 탐색한다.

- 표시 경로 = [`출발 스텝`] + [A^* 중간] + [`reverse(종단 스텝)`].
- 효과: 미세격자 탐색량을 근본 축소(랙↔랙 짧은 구간만 탐색) + 학습 스텝과 자동설계를 일치시킨다.

실측(project6 cell=100, 208 작업): 스텝 라우팅으로 199→206, 랙 집중도 17%→37%, 시간 단축.

7.2 진출/진입 면 학습(L2a/L3b)

스텝 폴백(매칭 배관 없음) 시, PoC 가 어느 면으로 빠지는지를 학습(EQUIP=-z·DUCT=+z 등)해 PoC 를 그

면으로 표면 투영한다(LiftPocToSurface). 다중면(bimodal) 키는 PoC 컨텍스트별 최근접(ANN) 표본으로

면을 분기하되, 자기검증 LOO 게이트(ANN 이 집계 다수결을 이기는 키만 채택)로 과적합을 방지한다.

7.3 접근불가 PoC 전처리(스냅)

파묻힌(솔리드 내부) PoC 는 학습면 방향 행진 + 반경확장으로 최근접 자유 셀에 스냅한다

(SnapPocToFreeCell). project6 cell=100 에서 +7 본 복구.

7.4 그룹배관 패턴(L4) — 공용 트렁크 랙 다발

같은 유틸리티 배관들은 동일 이격간격으로 평행하게 공용 트렁크(랙)를 다발로 탄다. 이를 탐지·학습해

신규 라우팅에 주입한다.

번들 탐지(Python bundle_detect):

8. **특징:** 방향론 압축, Arrow(Riser/Horizontal/Diagonal) 분류, 꺾임 수, 리샘플 방향벡터, extent.
9. **복합 유사도:** 형상 30%(Levenshtein) + 방향 30%(코사인) + 길이 20% + 규모 20%.
10. **(owner,util) pre-filter → Union-Find(임계 0.70) → 번들 게이트(꺾임≥2 + pitch CV≤0.30) → 트렁크 z·다발폭·pitch.**

학습 결과는 route_bundle_template(유틸별 대표 트렁크 고도 trunk_zs, 이격 pitch_mm)에 적재된다.

활용(UseBundlePattern, 기본 ON):

- **rack_levels(z)**: 학습 트렁크 고도를 엔진 **rack_levels**(회랑 페널티 면제 z-셀)로 주입 → 같은 유틸

새 배관이 공용 랙 높이에 모인다(강한 바이어스, 페널티 면제).

- **레인 회랑(xy)**: 트렁크 고도 ± 1 셀의 수평 런만 타이트한 회랑 셀로 주입(**BuildBundleCorridorCells**) →

충돌회피가 새 배관을 인접 레인에 분산(등간격 다발 패킹).

소프트 바이어스의 한계: 회랑은 비용 페널티(하드 레일 아님)라 z 추종은 강하나(rackZ 84%) xy 레인

추종은 약하다(번들밀집 2%). xy 추종을 회랑 가중으로 키우면 휴리스틱이 회랑 비용장을 과소평가해

탐색이 폭발한다. 이 한계가 7.5 절(폴리라인 복제)의 동기다.

7.5 기존배관 복제(폴리라인) — 가장 강한 설계 유사

소프트 바이어스의 느슨함을 넘어, 매칭되는 기존배관이 있으면 그 폴리라인을 그대로 복제하고

현재 점유에서 막힌(장애물이 달라진) 구간만 AW*로 국소 우회

수리한다(**UseDesignReplicate**, 기본 OFF).

알고리즘(**ReplicateMatchedPipes** / **ReplicatePath**):

11. 매칭 기존배관 폴리라인을 작업 PoC 방향에 맞춰 정렬, 양 끝에 실제 작업 PoC 부착.
12. 폴리라인을 **6-연결 셀 경로**로 변환(축정렬 구간을 셀 단위 행진 + 엘보/끝점은 ortho 보간).
13. 셀을 순회하며: 자유·인접 셀은 그대로 복제, 막힌 구간은 국소 AW*('RouteTask')로 우회 수리.
14. 수리 엔진 = 현재 장애물+설비만(다른 새 배관 마크 없음). 매칭 없음/수리 실패면 일반 AW* 결과 유지(폴백).
15. 거대격자(>30M 셀, RouteTask 의 Dense 복셀화 OOM 방지) 가드 — 셀을 키우라고 안내.

실측(CMP_KSCTA08, cell=50): ALKA 매칭 14/14 → 복제 14/14(수리 88 구간), ORG 2/2 → 복제 2/2

(수리 20 구간). 기존 폴리라인을 따라가고 새 장애물 만난 구간만 우회.

7.6 학습 활용 요약

자산	토글	기본	라우팅에 하는 일	엔진 변경
스텝(수직+엘보)	UseStubRouting	ON	출발/종단 스텝 고정, AW*는 스텝 끝~끝만	없음(시작/끝점)
진출/진입 면(L2a)	UsePatterns	ON	스텝 폴백 시 PoC 표면 투영	없음(시작/끝점)
그룹배관 패턴(L4)	UseBundlePattern	ON	트렁크 z(rack_levels)+레인 회랑	파라미터
기존설계 회랑(L2b)	UseDesignCorridor	OFF	매칭 폴리라인 회랑 시드	파라미터(w_corridor)
기존배관 복제	UseDesignReplicate	OFF	폴리라인 복제 + 막힌 곳만 수리	없음(후처리)

모든 학습 활용은 미적재/키 미스 시 기하 규칙으로 자동 폴백해 무해하다(회귀=실패 증가 없음).

8. C# 뷰어 — 시각화와 진단

8.1 워크플로

창 즉시 표시 → DB 비동기 로드(UI 비차단) → 프로젝트 선택(장애물만 표시) → 탐색 범위(모두/유틸그룹/유틸)

선택 실행 → 진행 다이얼로그 + 3D 라이브 오버레이.

8.2 3D 레이어(시각화)

레이어	색	내용
-----	---	----

복셀맵	회색 틀	격자 BBOX 프레임
점유맵	적색 큐브	voxelize 된 장애물 셀
방문맵	노랑 큐브	AW* 확장 셀(실패 진단)
최종경로	파랑 튜브(꺾임=녹색)	라우팅 경로(실측 관경)
기존 설계배관	유틸 색 튜브	TB_ROUTE_PATH 폴리라인
출발/중단 스텝	빨강/파랑	스텝(관경×1.35 반투명 셀)
그룹배관 패턴	보라 반투명 큐브	학습 트렁크 레인(관경×1.35)

자동(개발) 경로는 유틸 색을 밝게(Lighten) 그려 같은 유틸 기존배관(원색)과 구별한다.
스텝·패턴은 실제

관경보다 살짝 크게(×1.35) + 반투명으로 그려 배관 본체를 감싸는 셀로 보이게 한다.

8.3 진행 다이얼로그

네이티브 진행 콜백(`route_multi_progress`, phase 0=탐색진행 / 1=완료)을 받아 배관별
그룹/유틸·시작/중단

PoC·처리상태(%)·성공/실패·사유·확장수를 실시간 표로 보여준다. 행 클릭 시 그 배관의 로컬
미니 3D(복셀/

점유/방문/경로/패턴)와 결과 설명을 합성하고, 메인 뷰를 그 배관으로 줌한다.

실패 사유는 확장수로 추정: 확장수≤1=시작/끝 셀 막힘, ≥12M=탐색 상한(혼잡/막힘), 그
외=완전 차단.

8.4 헤드리스 진단(DbRouteDiag)

GUI 없이 빠른 A/B 반복을 위해 `--dbroute <proj> <cell> <util> <out> +`
환경변수(R3D_STUB/BUNDLE/

CORRIDOR/REPLICATE/WHEUR 등)로 각 학습 단계를 켜고 끄며
성공수·길이·랙집중도·번들밀집·복제 통계를

측정한다.

9. 성능과 검증

9.1 3 자 교차검증(Python = C++ = C#)

썬	결과
project6 cell=100(장애물 983·작업 208)	194/208 · 3,400,800mm — 3 자 완전 일치
project6 cell=200	multi 77 / rip-up 80(+3)
합성 혼잡(9×9 벽+툼 2)	seq 1/2 → ripup 2/2

9.2 학습 활용 누적 효과(project6 cell=100, 208 작업)

구성	성공	랙집중도	비고
전부 OFF(기하)	194~199	17%	베이스라인
+ 그룹 트렁크 회랑(L4)	207	39.7%	레인 다발
+ 스텝 라우팅	206	37%	랙↔랙 탐색
+ 동적 가중 AW*	208	—	완전 성공

9.3 정밀 셀 탐색량

스텝 라우팅 + ImplicitOccupancy + 동적 가중으로 미세격자 탐색량을 근본 해소:

project6 ALL 208 작업이 cell=50 0.14s, cell=25 0.5s, cell=10 7.7s(스텝 ON).

10. 개발 경위와 교훈

- **소프트 바이어스의 휴리스틱 함정:** 회랑 패널티(w_{corridor})를 강하게 주면 가중 AW* 휴리스틱(직선거리)이

회랑 비용장을 과소평가해 탐색이 Dijkstra 처럼 폭발한다. 혼잡 프로젝트에서 첫 어려운 배관이 탐색상한

초과로 실패했다. 해법은 회랑 가중을 안전값($\text{cell} \times 0.5$)으로 낮추되, 강한 설계추종이 필요하면 폴리라인

복제(7.5 절)로 전환하는 것이다.

- **z 추종 vs xy 추종의 비대칭:** rack_levels(z, 페널티 면제)는 강하게 따르지만 corridor(xy, 페널티)는

약하게 따른다. "패턴을 무시한다"는 인상의 원인이며, 하드 추종이 필요하다면 복제가 정답이다.

- **미세격자 가속은 병렬이 아니라 탐색량 축소:** 독립 배관 스레드 병렬화는 메모리대역 바운드로 이득이

없었다(시도·기각). 가속은 스텝·동적 가중·계층 corridor 로 탐색량 자체를 줄이는 게 정답.

- **무분별 ANN 은 해롭다:** 학습 면 분기를 무조건 ANN 으로 하면 회귀(199→192). 자기검증 LOO 게이트로

ANN 이 집계를 이기는 키만 채택해야 한다.

11. 결론과 향후 과제

현재 자동라우팅은 물리 타당성(충돌 0·관통 0) 과 설계 유사성(스텝·트렁크 다발·폴리라인 복제) 을

모두 달성한다. 점유 백엔드 3 종으로 셀 크기 무관하게 동작하고, 가중·동적 AW*로 혼잡 격자에서도 탐색

상한 내에 경로를 찾으며, 학습 활용은 모두 무해 풀백을 갖춘다.

향후 과제:

- **negotiated-congestion / CBS:** 비용기반 충돌 회피(rip-up 의 강력한 후속). 잔여 혼잡/막힘 실패 대상.
- **복제 + 다중배관 충돌 통합:** 복제 경로 간 충돌을 route_multi 마크와 통합(현재는 기존설계 무충돌 전제).
- **진출/진입 면 분기 정밀도:** 접근 세그먼트 방향열·인접 장애물 특징 추가.

문서 끝.